

Recursion

**"We achieve more
when we chase the
dream instead of
the competition."**

Simon Sinek



Good

Morning

Recursion - 1

TABLE OF CONTENTS

1. Why Recursion?
2. How to write recursive codes?
3. Sum of N natural numbers - Recursive function
4. Factorial of N - recursive function
5. N^{th} Fibonacci Number
6. Time and Space Complexity of Recursive codes



Notes



Why Recursion?

- Pre-requisite of Backtracking , Trees , D.P , Graphs
- Sorting algo's [Merge , Sort , Quick Sort]

Steps to follow for Recursive code

01. Assumption / Expectation → Decide what your function is suppose to do.
02. Main logic → Breaking the problem into sub problem & using it to solve larger problem
- 03 Base case → Last valid input for which the recursion has to stop.



Recursion

- Function calling itself
- Repetition of same task
- Repetitive using the same pattern

↳ Solving a bigger problem using the smaller instances of the same problem

subproblem

Sum of first N natural no's

$$\text{sum}(5) = \underbrace{1 + 2 + 3 + 4}_{\text{sum}(4)} + 5$$

$$\text{sum}(5) = \underbrace{\text{sum}(4)}_{\text{subproblem}} + 5$$

$$\text{sum}(n) = \underbrace{1 + 2 + 3 + 4 + \dots + (n-1)}_{\text{subproblem}} + n$$

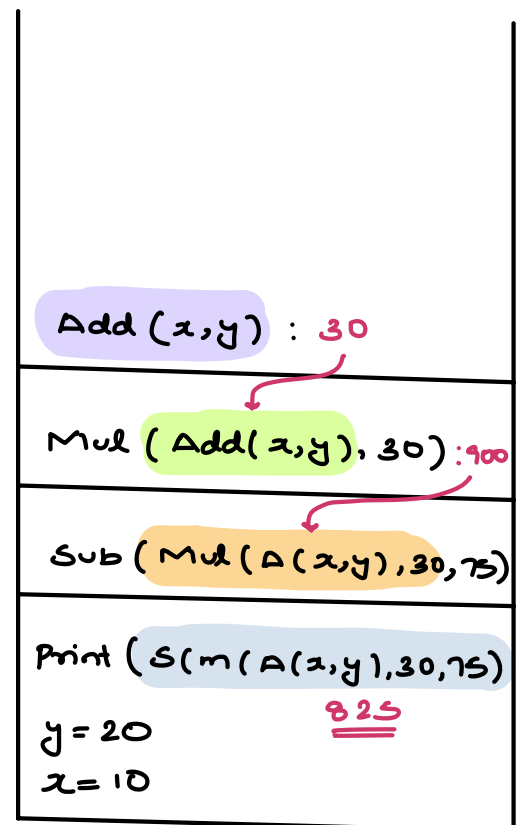
$$\text{sum}(n) = \text{sum}(n-1) + n$$



Function Call Tracing

Code-block

```
int add ( int x, int y ){  
    return x + y ;  
}  
  
int mul ( int x, int y ){  
    return x * y ;  
}  
  
int sub ( int x, int y ){  
    return x - y ;  
}  
  
void print ( int x ){  
    print (x) ;  
}
```



main () {

x = 10, y = 20

print (sub (mul (add (x,y), 30), 75); = 825

↳ (mul (add(x,y), 30) = 900

↳ add(x,y) = 30

Ans = 825

01. Function will get added at the top.
02. Once a child call is executed, then only parent call will resolve.



// Assumption → Calculate & return sum from 1 to N

```
int sum (int N){  
    if (N == 1) return 1;  
    return sum(N-1) + N;  
}
```

sum(1): return	1
sum(1) + 2	: 3
sum(2) + 3	: 6
sum(3) + 4	: 10
sum(4)	: 10

N Factorial = Given N (whole number) , find the factorial of N

$$\text{fact}(5) = 5 * \underbrace{4 * 3 * 2 * 1}_{\text{fact}(4)} = 120$$

$$\text{fact}(n) = n * \underbrace{(n-1) * (n-2) * \dots * 3 * 2 * 1}_{\text{fact}(n-1)}$$

$N > 0$

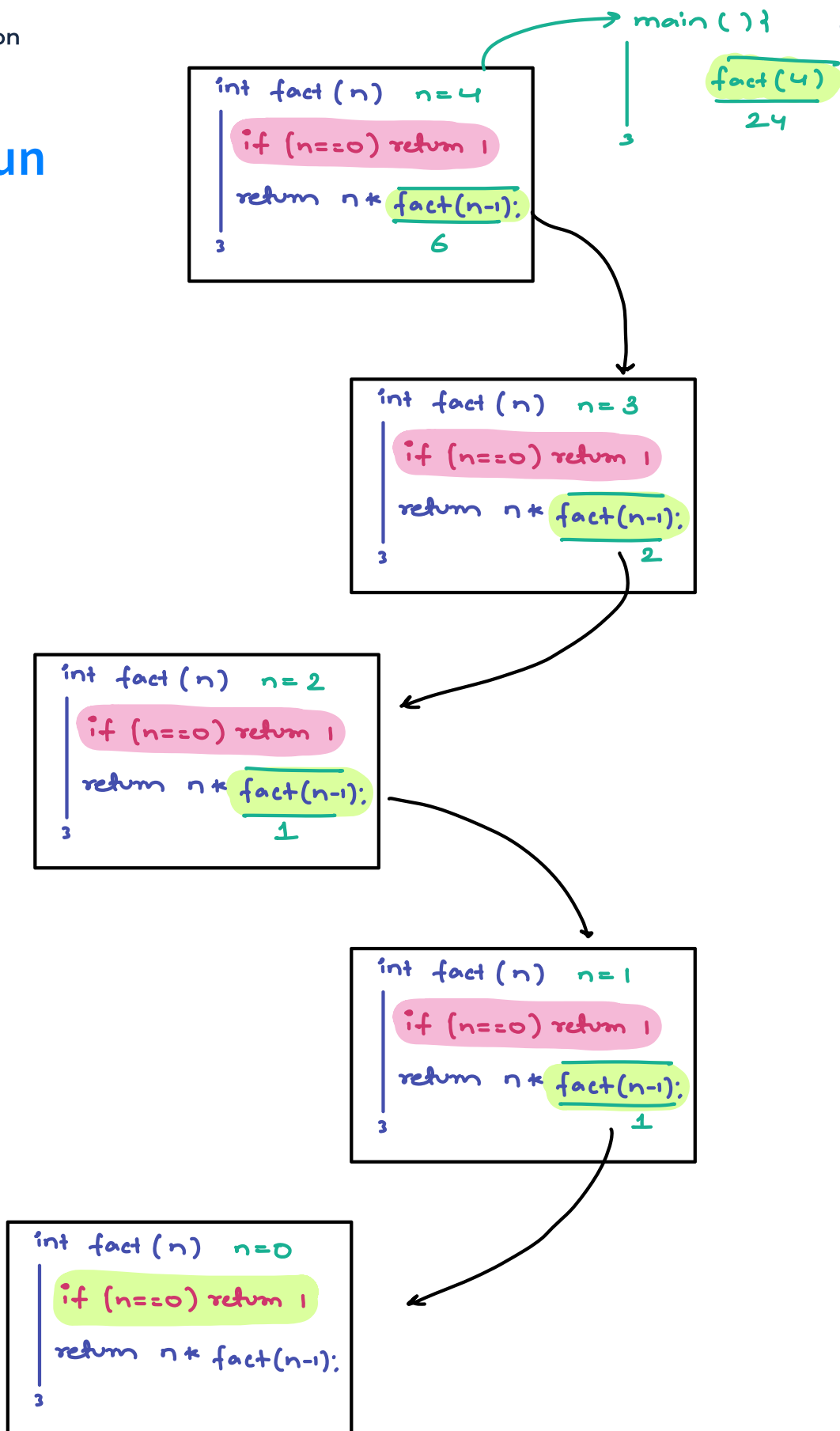
```
int fact(n)  
|  
| if (n == 1) return 1  
|  
| return n * fact(n-1);  
|  
3
```

$N \geq 0$

```
int fact(n)  
|  
| if (n == 0) return 1  
|  
| return n * fact(n-1);  
|  
3
```



dry-run





Nth Fibonacci

N =	0	1	2	3	4	5	6	7	8	9	10
	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
F.b →	0	1	1	2	3	5	8	13	21	34	55

Given value of N. Write a recursive function of find Nth fibonacci number.

$\text{fib}(n) = \text{sum of previous two fib no.}$

$$\text{fib}(5) = \text{fib}(4) + \text{fib}(3)$$

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

// Assumption → Calculate & return nth fib no.

```
int fib(int N)
{
    if (N == 0) return 0;
    if (N == 1) return 1;
    return fib(N-1) + fib(N-2);
}
```

if (N ≤ 1) return N

Base case = Last valid input for which recursion has to stop.

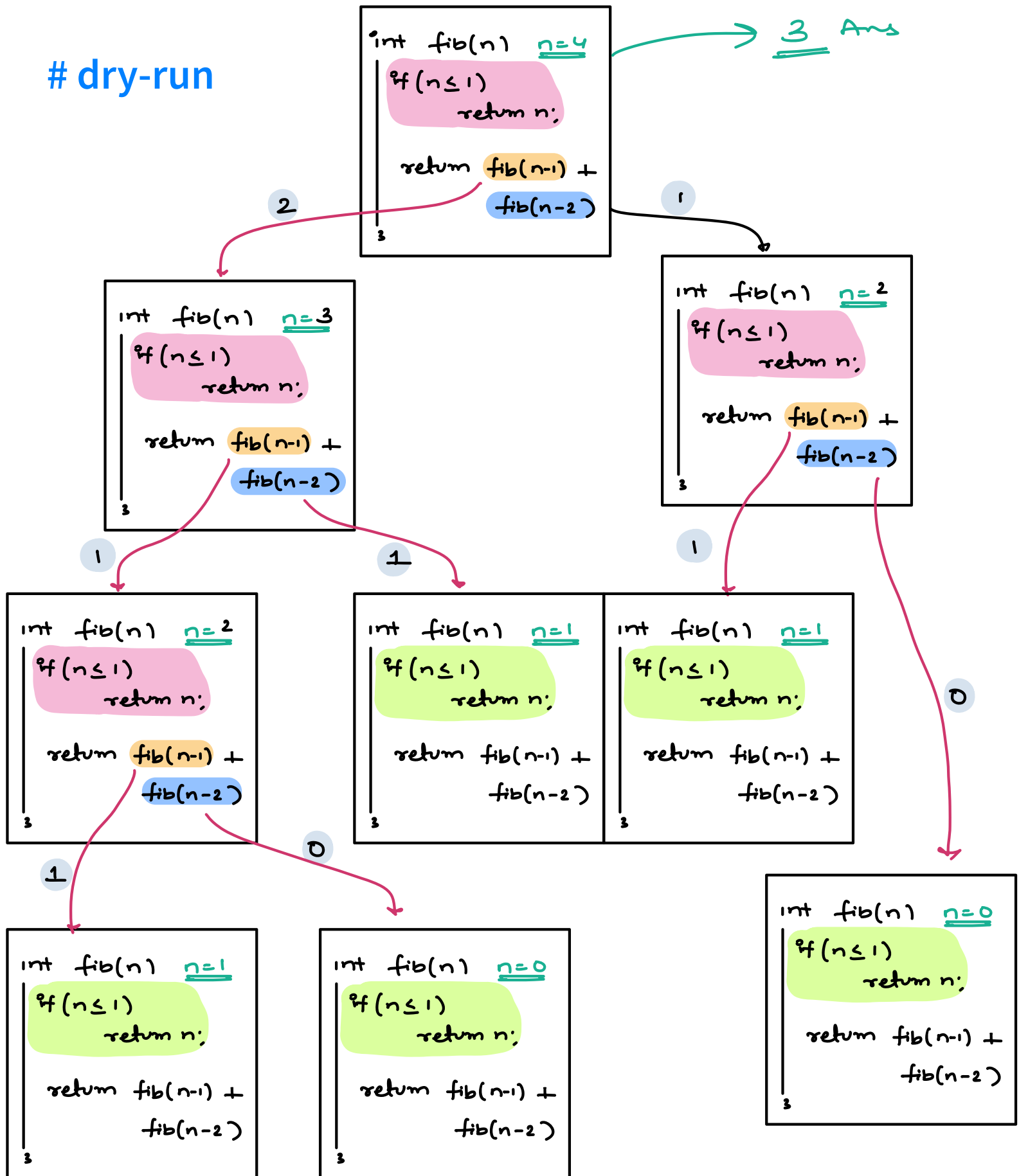
$$N=0 \quad \text{fib}(-1) + \text{fib}(-2)$$

$$N=1 \quad \text{fib}(0) + \text{fib}(-1)$$

$$N=2 \quad \text{fib}(1) + \text{fib}(0)$$



dry-run





Time Complexity of Recursion - Using Recurrence relation

```
int fact (int N){
    if (N==0) {return 1}
    return fact(N-1)*N;
}
```

$T(N-1)$

$T(N) \rightarrow$ time taken to calculate factorial of N .

time taken to calculate factorial of $N-1 \Rightarrow ?$ $T(N-1)$

$$T(n) = T(n-1) + 1 = \text{Recurrence relation}$$

$$T(n) = T(n-1) + 1 \rightarrow 1^{\text{st}} \text{ substitution}$$

$$T(n-1) = T(n-2) + 1$$

$$T(n) = T(n-2) + 1 + 1$$

$$T(n) = T(n-2) + 2 \rightarrow 2^{\text{nd}} \text{ substitution}$$

$$T(n-2) = T(n-3) + 1$$

$$T(n) = T(n-3) + 1 + 2$$

$$T(n) = T(n-3) + 3 \rightarrow 3^{\text{rd}} \text{ subs}$$

After K^{th} substitution

$$T(n) = T(n-K) + K$$

$$T(0) = 1$$

$$n-K=0$$

$$K=n$$

$$T(n) = T(0) + n$$

$$T(n) = 1 + n \approx n$$



$$T_c = O(n)$$

$$SC: O(n)$$

T.C Fibonacci

```
int fib(int N){
    if(N ≤ 1) {return N}
    return fib(N-1) + fib(N-2);
}
```

$\underbrace{\text{fib}(N-1)}_{T(N-1)} + \underbrace{\text{fib}(N-2)}_{T(N-2)}$

Recurrence relation →

// Assumption = fib(N) is going to take T(N) time

$$T(N) = T(N-1) + T(N-2) + 1$$

$$T(N) = T(N-1) + T(N-1) + 1$$

$$T(N) = 2 * T(N-1) + 1 = 2^1 T(N-1) + 2^1 - 1$$

$$T(N-1) = 2 * T(N-2) + 1$$

$$T(N) = 2 * (2 * T(N-2) + 1) + 1$$

$$T(N) = 4 T(N-2) + 3 = 2^2 T(N-2) + 2^2 - 1$$

$$T(N-2) = 2 * T(N-3) + 1$$

$$T(N) = 4 * (2 * T(N-3) + 1) + 3$$

$$= 8 T(N-3) + 7 = 2^3 T(N-3) + 2^3 - 1$$

After k substitution

$$T(n) = 2^k T(n-k) + 2^k - 1$$

$$T(1) = 1$$

$$n - k = 1$$

$$k = n - 1$$

$$T(n) = 2^{n-1} * 1 + 2^{n-1} - 1$$

$$= 2^{n-1} + 2^{n-1} - 1$$

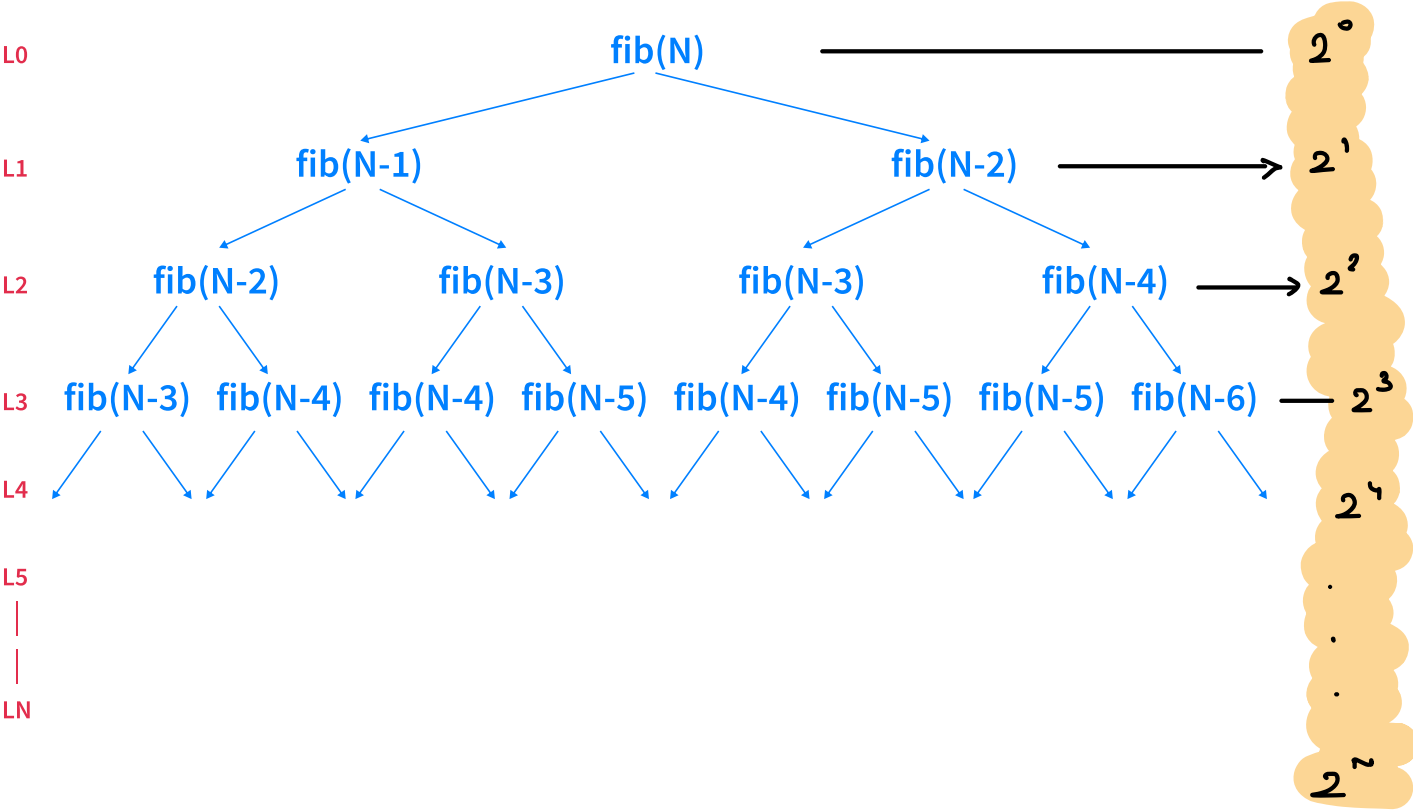
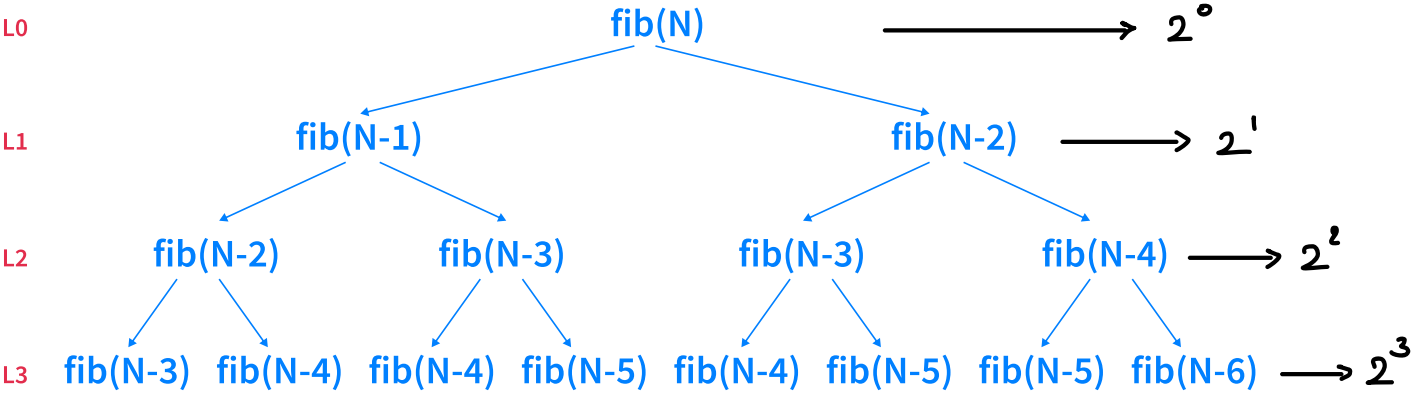
$$= 2^n - 1$$

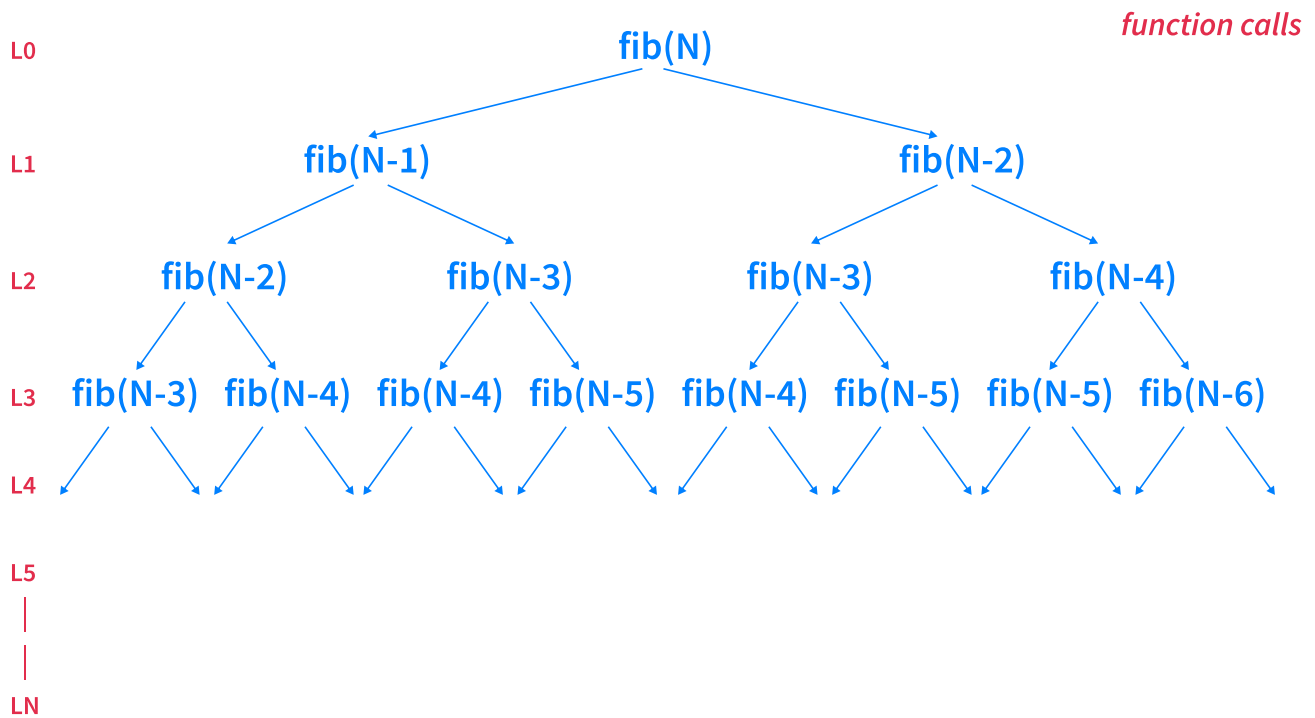
$$T(n) \leq O(2^n)$$

* Another definition of Time complexity

TC = Time taken in a single fn call *

No. of function calls





total function calls :

$$\text{Total calls} = 2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^n$$

$$\text{Sum of GP} = \frac{a(r^n - 1)}{r - 1}$$

$$= \frac{2^0 * (2^{n+1} - 1)}{2 - 1}$$

$$= (2^{n+1} - 1)$$

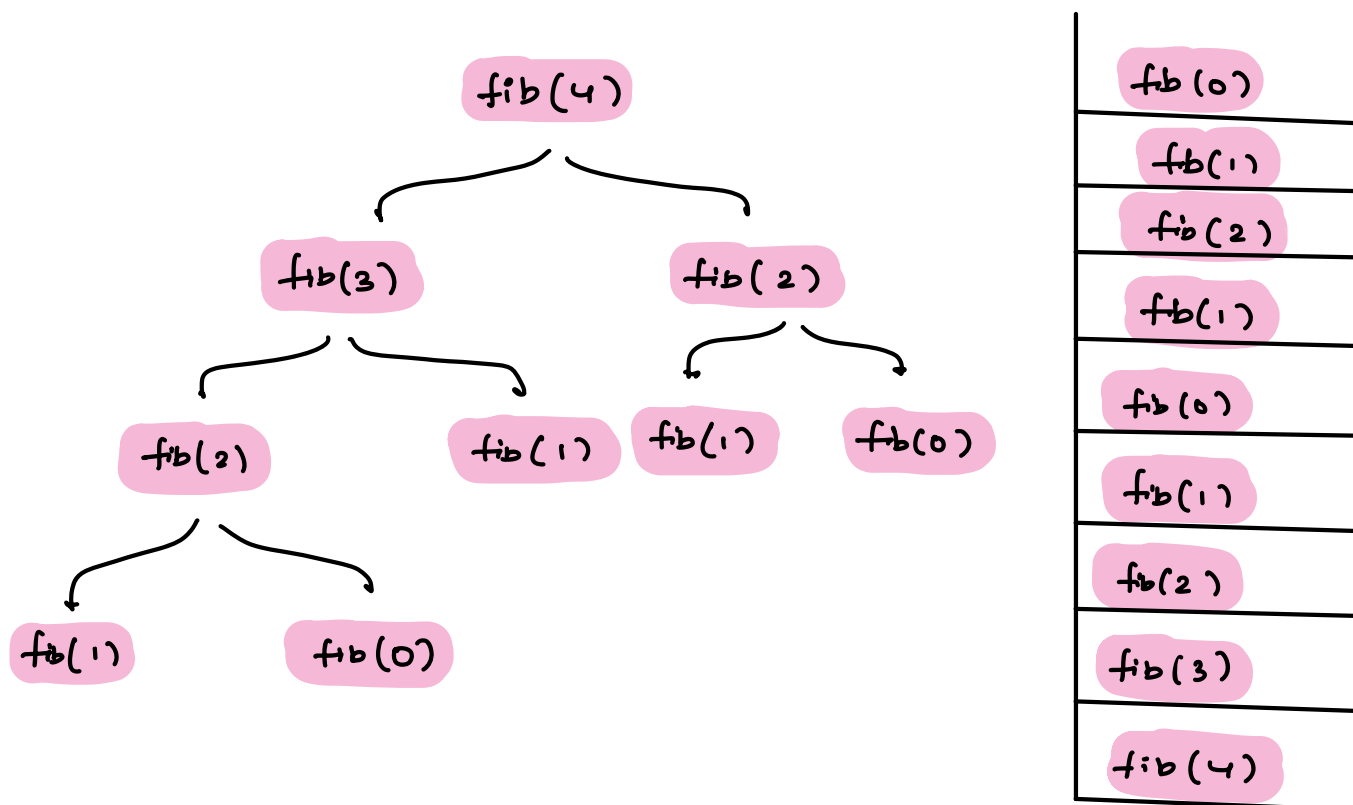
$$TC = (2^{n+1} - 1) * 1$$

$$= O(2^n)$$



Space Complexity

⇒ Max Amount of calls we put in our stack at any point of time is our space complexity of recursive function.

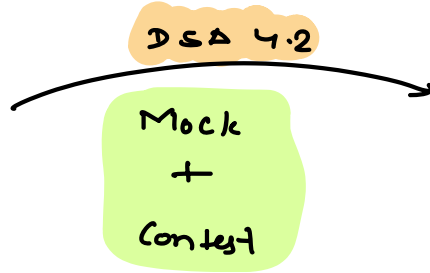
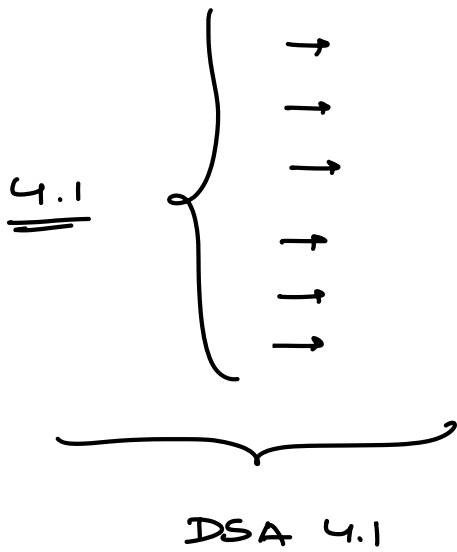


At max, 4 calls

$Tc: O(2^n)$

$Sc: O(n)$

DSA 4.2



- Backtracking
- Inverse modulo
- String pattern matching
- DP on strings
- Graphs Algo

_____ α _____ α _____ α _____

Subarray (OR)

$$\begin{array}{lcl}
 1 & = & 0 \ 0 \ 1 \\
 2 & = & 0 \ 1 \ 0 = \frac{1 * (1+1)}{2} \\
 3 & = & 0 \ 1 \ 1 \\
 4 & = & 1 \ 0 \ 0 = \frac{1 * (1+1)}{2}
 \end{array}$$

{1, 2, 3, 4}

$$\{1\} = 0 \ 0 \ 1 \quad \frac{n * (n+1)}{2}$$

$$\{1, 2\} = 0 \ 1 \ 1$$

$$\{1, 2, 3\} = 0 \ 1 \ 1$$

$$\{1, 2, 3, 4\} = 1 \ 1 \ 1$$

$$\{2\} = 0 \ 1 \ 0$$

$$\{2, 3\} = 0 \ 1 \ 1$$

$$\{2, 3, 4\} = 1 \ 1 \ 1$$

$$\begin{array}{l}
 \{1, 3, 4, 6\} \\
 + \\
 \{1, 2, 3, 4, 5, 6\}
 \end{array}$$

$$\{3\} = 011$$

$$\{3, 4\} = 111$$

$$\{4\} = 100$$

```
for (i=0; i<n; i++) {
```

```
    xor = xor ^ A[i]
```

```
}
```

```
for (i=1; i<=n+2; i++) {
```

```
    xor = xor ^ i;
```

```
}
```

xor = xor of two distinct ele

Min XOR value $\Rightarrow$ Sorting can help

$\{2, 10, 6, 8\}$

$\{2, 6, 8, 10\}$

$$2^6$$

$$6^8$$

$$8^{10}$$

} Answer will lie
in these three option